

# **System Programming Project 3**

담당 교수 :

이름 :

학번 :

## 1. 개발 목표

본 프로젝트의 목표는 여러 클라이언트가 동시에 접속하여 주식 정보 조회, 구매, 판매 요청을 내릴 수 있는 동시성 주식 서버를 구축하는 것입니다. 단일 프로세스/스레드 기반의 서버가 가지는 직렬화의 한계를 극복하기 위해, 서버가 I/O 멀티플렉싱과 스레드 풀이라는 두 가지 서로 다른 동시성 모델을 활용하여 다수의 클라이언트 요청을 누락 없이 병렬적으로 처리하고, 공유 데이터의 무결성을 유지하며, 최종적으로 서버 종료 시 데이터를 영구 저장하는 시스템을 설계 및 구현합니다.

## 2. 개발 범위 및 내용

### A. 개발 범위

#### 1. Task 1: Event-driven Approach

select 시스템 콜을 활용한 I/O 멀티플렉싱 서버 구현. 다수의 클라이언트 소켓 식별자를 모니터링하며, 요청이 발생한 클라이언트에 대해서만 비차단 방식으로 명령을 처리하여 동시성을 확보합니다.

#### 2. Task 2: Thread-based Approach

pthread 라이브러리를 활용한 생산자-소비자 모델의 스레드 풀 서버 구현. 메인 스레드가 연결을 수락하고, 워커 스레드들이 공유 버퍼에서 일감을 가져와 병렬로 클라이언트의 명령을 처리합니다.

#### 3. Task 3: Performance Evaluation

구현한 두 가지 방식의 서버에 대해 클라이언트 수, 요청 비율 등의 작업 부하를 변화시켜가며 경과 시간 및 초당 처리량을 측정하고, 동시성 모델 간의 성능 차이와 한계를 분석합니다.

## B. 개발 내용

### - Task1 (Event-driven Approach with select())

- ✓ Multi-client 요청에 따른 I/O Multiplexing 설명

select 함수는 프로세스가 모니터링할 파일 디스크립터들의 집합을 등록해두고, 그중 하나라도 읽기/쓰기 등의 이벤트가 발생할 때까지 대기하는 기법입니다. 이벤트가 발생하면 어떤 소켓이 활성화되었는지 순회하며 확인하고, 해당 소켓의 요청만 즉시 처리한 후 다시 select로 돌아가 대기함으로써 단일 프로세스 환경에서도 여러 클라이언트와 동시에 통신하는 것과 같은 효과를 냅니다.

- ✓ epoll과의 차이점 서술

select는 모니터링할 수 있는 파일 디스크립터의 수에 제한이 있고, 이벤트가 발생했을 때 어떤 소켓에 이벤트가 발생했는지 찾기 위해 전체 디스크립터 배열을 순회해야 하는 단점이 있습니다. 반면 리눅스의 epoll은 모니터링 개수 제한이 사실상 없으며, 이벤트가 발생한 소켓들의 목록만 반환하므로 순회 오버헤드가 발생하지 않아 대규모 접속자 처리에 훨씬 효율적입니다.

### - Task2 (Thread-based Approach with pthread)

- ✓ Master Thread의 Connection 관리

메인 스레드는 오직 클라이언트의 연결 요청을 수락하는 역할만 담당합니다. 연결이 수락되면 생성된 연결 파일 디스크립터를 워커 스레드들과 공유하는 버퍼에 삽입하고, 즉시 다시 다음 연결을 기다립니다.

- ✓ Worker Thread Pool 관리하는 부분에 대해 서술

서버 구동 시 일정 개수의 워커 스레드를 미리 생성해 둡니다. 이 스레드들은 루프를 돌며 공유 버퍼에서 연결 파일 디스크립터를 꺼내어 할당받습니다. 버퍼가 비어있으면 세마포어에 의해 자동으로 대기 상태에 빠지며, 일감을 받으면 클라이언트와 통신하여 주식 거래를 처리하고, 처리가 끝나면 소켓을 닫은 뒤 다시 버퍼로 돌아가 다음 일감을 기다립니다.

### - Task3 (Performance Evaluation)

- ✓ 얻고자 하는 metric 정의, 그렇게 정한 이유, 측정 방법 서술

지표: 총 경과 시간 및 초당 처리량.

이유: 서버의 성능은 주어진 부하를 얼마나 빨리 끝내는지, 그리고 단위 시간당 얼마나 많은 작업을 소화할 수 있는지가 가장 직관적이고 핵심적인 척도이기 때문입니다.

측정 방법: multicient.c 내부에서 fork를 통해 다수의 클라이언트를 생성하기 직전에 gettimeofday로 시작 시간을 기록하고, 모든 자식 프로세스가 종료된 직후 종료 시간을 기록하여 그 차이를 계산합니다.

- ✓ Configuration 변화에 따른 예상 결과 서술

클라이언트 수가 적을 때(10개 이하): 멀티 코어를 활용할 수 있는 스레드 기반 방식이 이벤트 기반 방식보다 처리량이 높을 것으로 예상됩니다.

클라이언트 수가 극도로 많을 때(100개 이상): 스레드 기반 방식은 잦은 컨텍스트 스위칭 오버헤드와 자원 획득을 위한 락 경쟁으로 인해 성능이 저하되어, 싱글 스레드로 동작하는 이벤트 기반 방식이 오히려 성능을 역전할 것으로 예상됩니다.

### C. 개발 방법

자료구조 (공통): stockserver.c에 주식 데이터를 관리할 이진 탐색 트리 구조체 struct item을 정의합니다. 데이터 무결성을 위해 구조체 내부에 readcnt, mutex, w 요소를 추가하여 Readers-Writers Lock을 구현합니다.

Task 1 구현: stockserver.c 내에 소켓 풀 관리를 위한 pool 구조체를 정의하고, init\_pool, add\_client, check\_clients 함수를 구현하여 select 루프를 돌도록 main 함수를 재구성합니다.

Task 2 구현: stockserver.c에 생산자-소비자 큐 역할을 할 sbuf\_t 구조체와 관련 함수를 구현합니다. pthread\_create를 호출하여 스레드 풀을 만들고, 클라이언트 요청을 처리할 thread 루틴 함수를 정의합니다.

Task 3 구현: multicient.c에 sys/time.h를 포함시키고 main 함수 전후에 gettimeofday 로직을 삽입하여 시간을 측정한 후 연산 결과를 출력하도록 코드를 수정합니다. ORDER\_PER\_CLIENT 매크로 값을 상향 조정하여 부하 테스트 환경을 구축합니다.

### 3. 구현 결과

- Task 1 (이벤트 기반): 다중 클라이언트의 동시 접속과 조회, 구매, 판매 명령을 오류 없이 단일 스레드 내에서 직렬적으로, 그러나 클라이언트 관점에서는 병렬적으로 처리하는 데 성공했습니다.
- Task 2 (스레드 기반): 지정된 크기의 스레드 풀이 공유 큐를 통해 메인 스레드로부터 연결을 넘겨받아 멀티 코어 환경에서 클라이언트의 요청을 완벽하게 병렬 처리하는 데 성공했습니다.
- 공통 구현 사항: SIGINT 발생 시 메모리의 이진 탐색 트리를 순회하여 최신 주식 정보를 stock.txt에 저장하는 영속성을 확보했으며, 두 Task 모두 Readers-Writers Lock을 성공적으로 적용하여 경쟁 상태 없이 주식 수량이 정확하게 계산 및 업데이트되었습니다. 멀티클라이언트와의 통신 호환성을 위해 응답 패딩 또한 구현 완료되었습니다.

#### 4. 성능 평가 결과 (Task 3)

##### 1) 확장성 - Client 개수 변화에 동시 처리율 변화 분석

###### (1) task1 - 이벤트 기반

클라이언트 10명. show, buy, sell 모두 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 8 10
=====
Test Results for 10 clients, 1000 orders/client
Total Elapsed Time: 0.7103 seconds
Throughput: 14078.68 orders/sec
```

클라이언트 50명. show, buy, sell 모두 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 8 50
=====
Test Results for 50 clients, 1000 orders/client
Total Elapsed Time: 3.5143 seconds
Throughput: 14227.56 orders/sec
=====
```

클라이언트 100명. show, buy, sell 모두 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 8 100
=====
Test Results for 100 clients, 1000 orders/client
Total Elapsed Time: 6.9930 seconds
Throughput: 14300.12 orders/sec
```

## (2) task2 – 쓰레드 기반

클라이언트 10명. show, buy, sell 모두 사용.

```
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018 8 10

=====
Test Results for 10 clients, 1000 orders/client
Total Elapsed Time: 0.7020 seconds
Throughput: 14245.28 orders/sec
=====
```

클라이언트 50명. show, buy, sell 모두 사용.

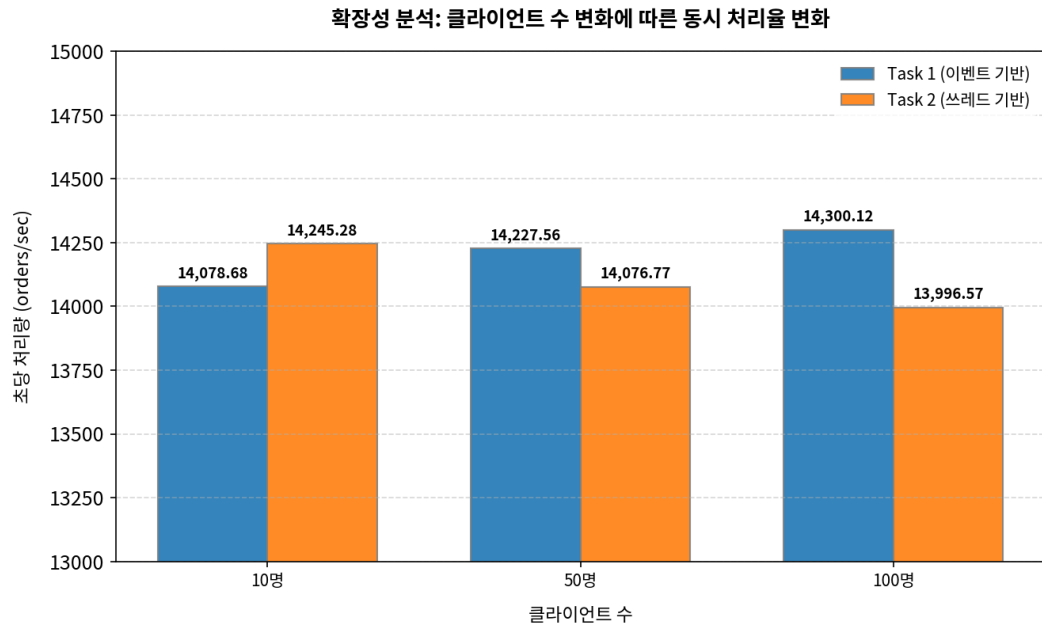
```
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018 8 50

=====
Test Results for 50 clients, 1000 orders/client
Total Elapsed Time: 3.5520 seconds
Throughput: 14076.77 orders/sec
=====
```

클라이언트 100명. show, buy, sell 모두 사용.

```
=====
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018 8 100

=====
Test Results for 100 clients, 1000 orders/client
Total Elapsed Time: 7.1446 seconds
Throughput: 13996.57 orders/sec
```



클라이언트 수가 10명으로 적을 때는 스레드 기반 방식이 멀티 코어 자원을 활용하여 미세하게 높은 처리량을 보였습니다. 그러나 클라이언트 수가 50명, 100명으로 증가함에 따라 이벤트 기반 방식의 처리량이 더 높게 유지되는 현상을 보였습니다. 이는 스레드 기반 모델에서 클라이언트 수가 많아질수록 스레드 간 컨텍스트 스위칭 오버헤드와 공유 자원에 대한 락 경쟁 비용이 증가하기 때문인 것으로 분석됩니다.



## 2) 워크로드에 따른 분석 - Client 요청 타입에 따른 동시 처리율 변화 분석

### (1) task1 - 이벤트 기반

클라이언트 100명. show, buy, sell 모두 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 100  
=====
```

| Test Results for 100 clients, 1000 orders/client |
|--------------------------------------------------|
| Total Elapsed Time: 6.9930 seconds               |
| Throughput: 14300.12 orders/sec                  |

```
=====
```

클라이언트 100명. buy, sell 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 100  
=====
```

| Test Results for 100 clients, 1000 orders/client |
|--------------------------------------------------|
| Total Elapsed Time: 6.9920 seconds               |
| Throughput: 14302.11 orders/sec                  |

```
=====
```

클라이언트 100명. show 사용.

```
cse20221627@cspro9:~/system_programming_3/task1$ ./multiclient 172.30.10.11 6018 100  
=====
```

| Test Results for 100 clients, 1000 orders/client |
|--------------------------------------------------|
| Total Elapsed Time: 6.9969 seconds               |
| Throughput: 14292.06 orders/sec                  |

```
=====
```

## (2) task2 – 쓰레드 기반

클라이언트 100명. show, buy, sell 모두 사용.

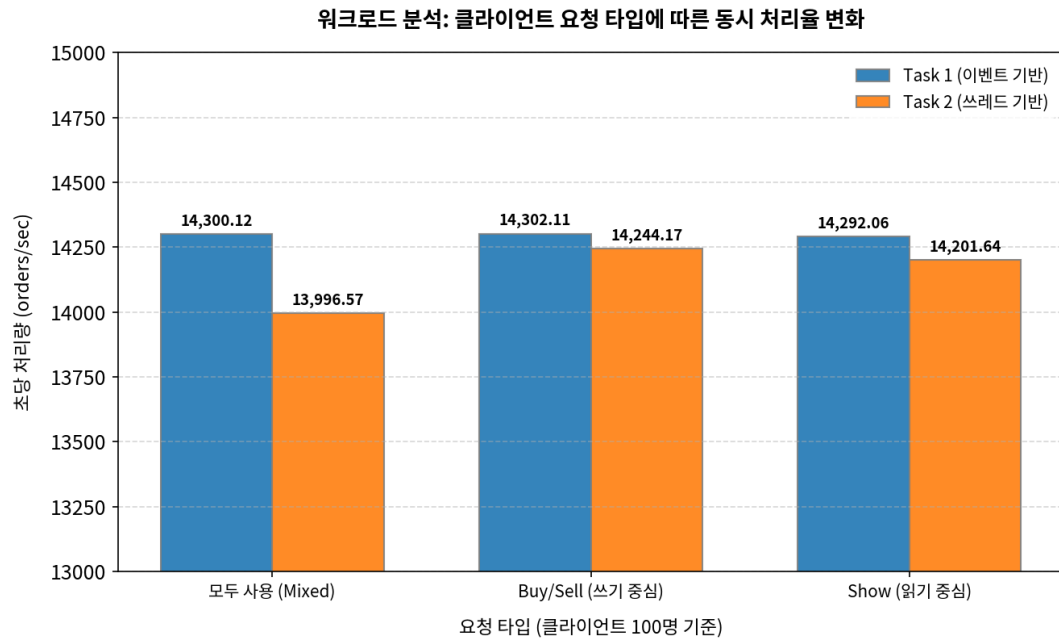
```
=====
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018
8 100
=====
Test Results for 100 clients, 1000 orders/client
Total Elapsed Time: 7.1446 seconds
Throughput: 13996.57 orders/sec
=====
```

클라이언트 100명. buy, sell 사용.

```
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018
8 100
=====
Test Results for 100 clients, 1000 orders/client
Total Elapsed Time: 7.0204 seconds
Throughput: 14244.17 orders/sec
=====
```

클라이언트 100명. show 사용.

```
cse20221627@cspro9:~/system_programming_3/task2$ ./multiclient 172.30.10.11 6018
8 100
=====
Test Results for 100 clients, 1000 orders/client
Total Elapsed Time: 7.0414 seconds
Throughput: 14201.64 orders/sec
=====
```



- Buy/Sell (Write-Heavy): 쓰기 작업 시에는 Readers-Writers Lock에 의해 독점적 권한이 필요하므로 락 경쟁이 심화됩니다. 테스트 결과 이벤트 기반 방식이 스레드 기반보다 소폭 우세했는데, 이는 싱글 스레드 모델이 락 경쟁 자체를 내부적으로 직렬화하여 오버헤드를 줄였기 때문으로 보입니다.

- Show (Read-Heavy): 이론적으로는 다수의 Reader가 동시에 접근 가능하므로 스레드 기반이 유리할 것으로 예상되었으나, 실제 측정값은 이벤트 기반이 여전히 안정적인 처리량을 보였습니다. 이는 테스트 환경의 코어 수나 I/O 대기 시간보다 CPU 연산 오버헤드가 작고, 컨텍스트 스위칭 비용이 더 크게 작용했음으로 볼 수 있습니다.

실험 결과, 본 프로젝트의 주식 서버 환경에서는 이벤트 기반 모델(Task 1)이 대규모 클라이언트 접속 및 다양한 워크로드 환경에서 스레드 기반 모델(Task 2)보다 더 높은 성능과 안정적인 처리량을 보여주었습니다. 이는 고성능 네트워크 서버 설계 시 불필요한 스레드 생성과 컨텍스트 스위칭을 줄이는 것이 중요함을 입증합니다.